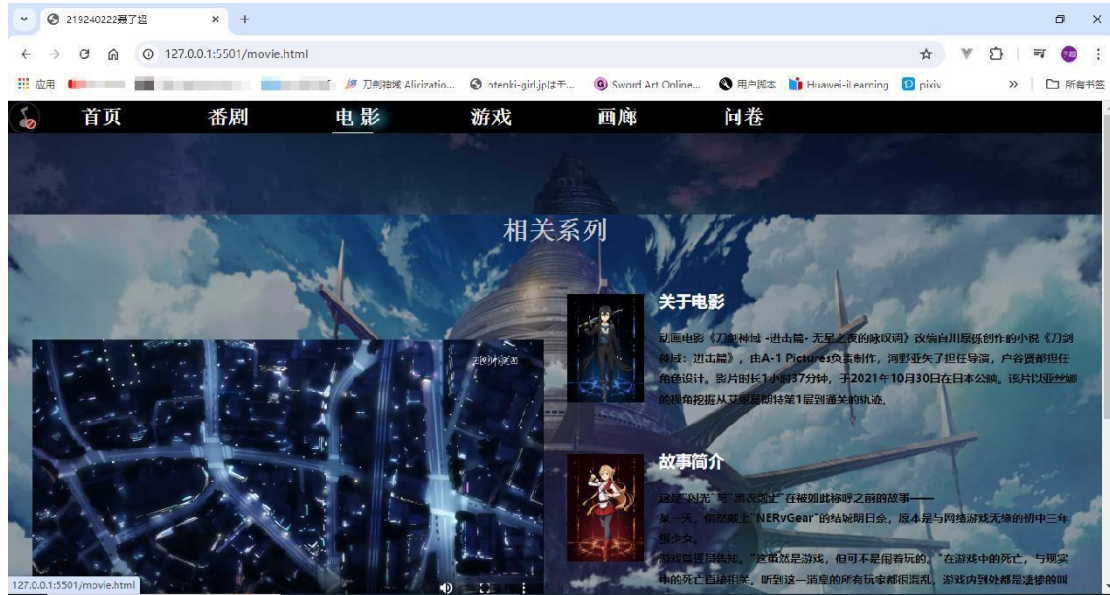
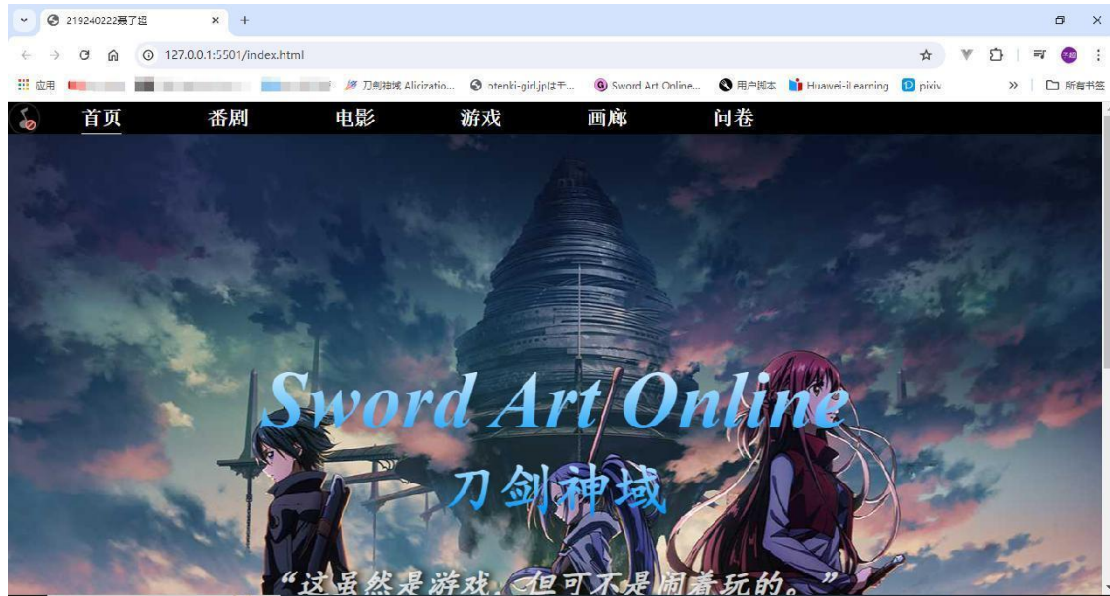


项目前端网页展示

原 html 所有页面(未修改前)



219240222 累了超

127.0.0.1:5501/questionnaire.html

应用 刀剑神域 <div>... </div> stenki-girl.jpは平... Sacred Art Online... 用户脚本 Huawei Learning pidv 所有书签

首页 番剧 电影 游戏 画廊 问卷

问卷调查

性别: ☒ 男 ☐ 女

年龄:

联系邮箱:

联系电话:

你接触过ACG文化

219240222 累了超

127.0.0.1:5501/questionnaire.html

应用 刀剑神域 <div>... </div> stenki-girl.jpは平... Sacred Art Online... 用户脚本 Huawei Learning pidv 所有书签

联系电话:

你接触过ACG文化: ☒ 接触过 ☐ 听过但没接触过 ☐ 没接触过

您看过或听过哪些刀剑系列的动漫或者游戏?

☐ 刀剑神域第一季	☐ 刀剑神域第二季
☐ 刀剑神域第三季	☒ 刀剑神域剧场版序列之争
☐ 刀剑神域外传	☒ 刀剑神域记忆碎片
☐ 刀剑神域AUS/UB	☐ 刀剑神域IF
☐ 没看过, 没听过	

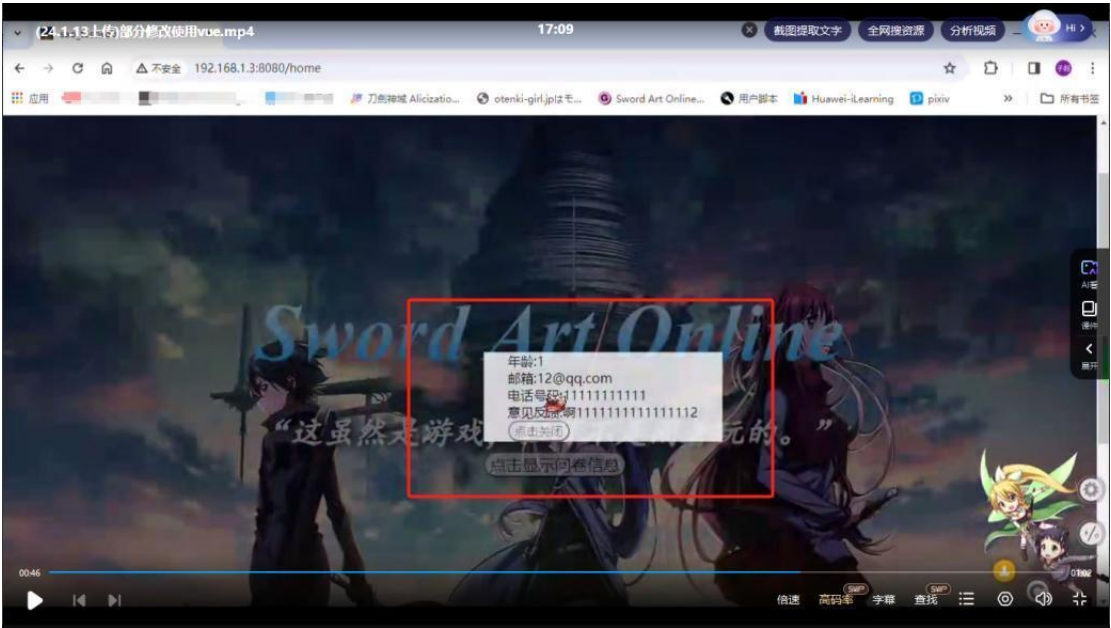
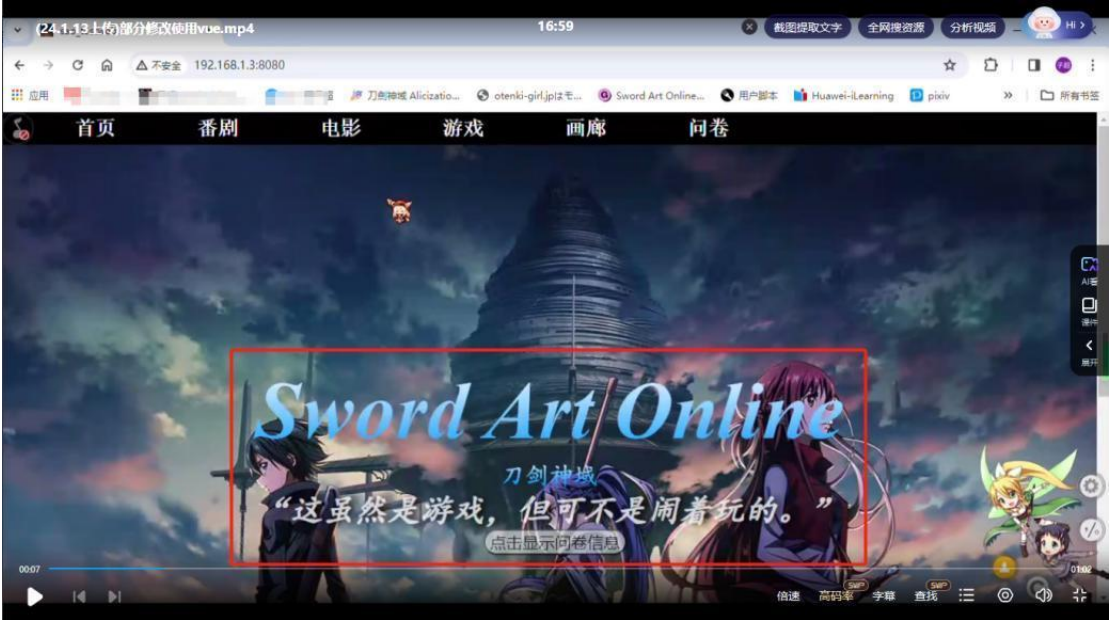
对该网页有什么改进或建议:

若无改进或建议填无

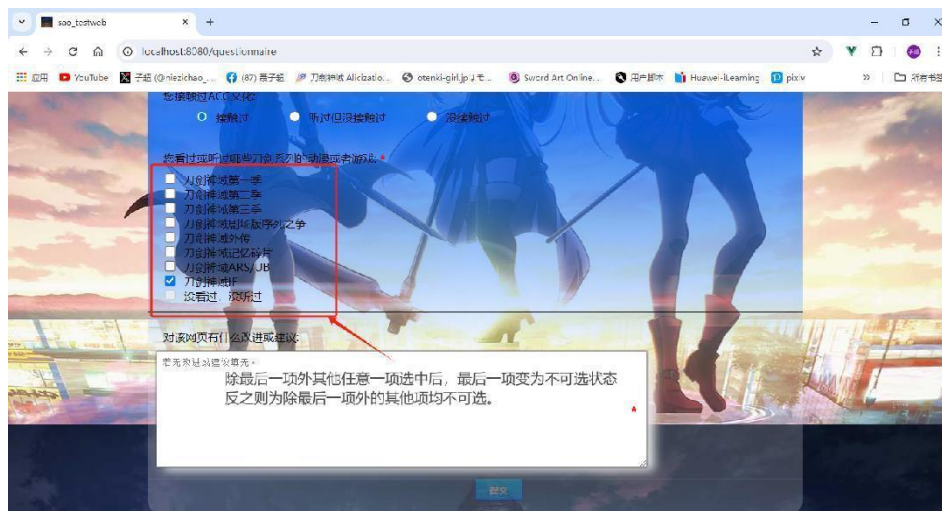
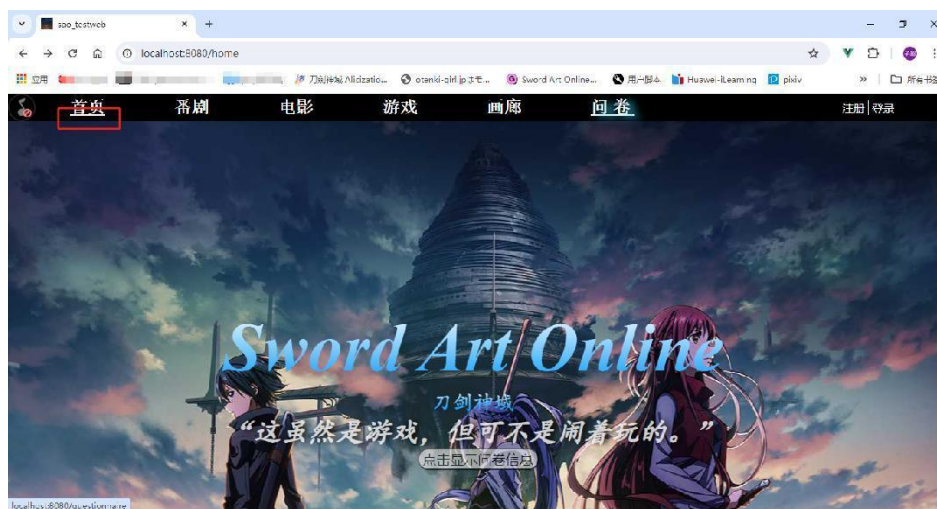
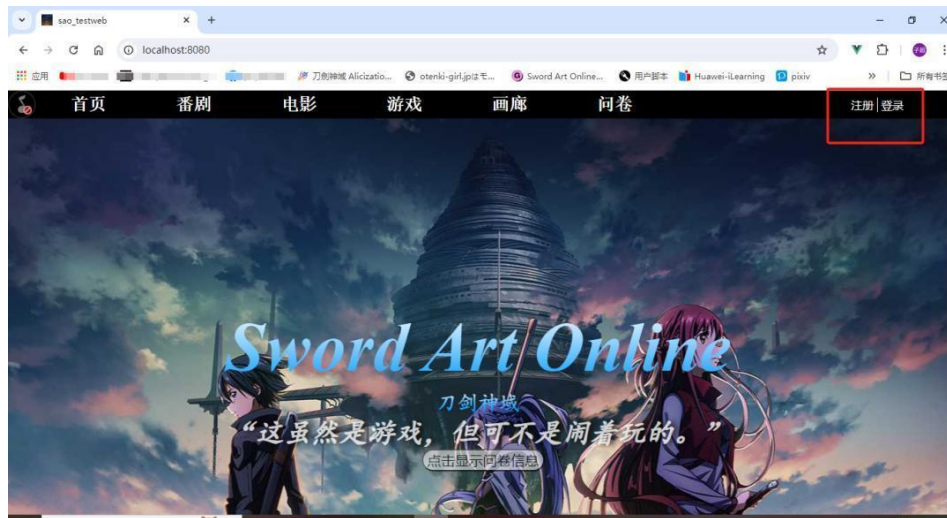
提交

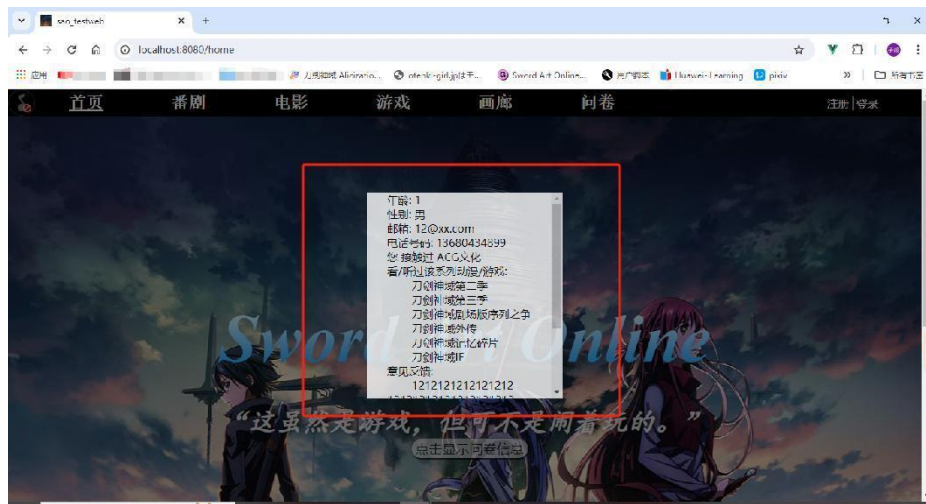
修改成 vue 项目并添加功能

(此处展示新添加部分，多个单独的 html 改为由组件渲染的，在下面有截图)

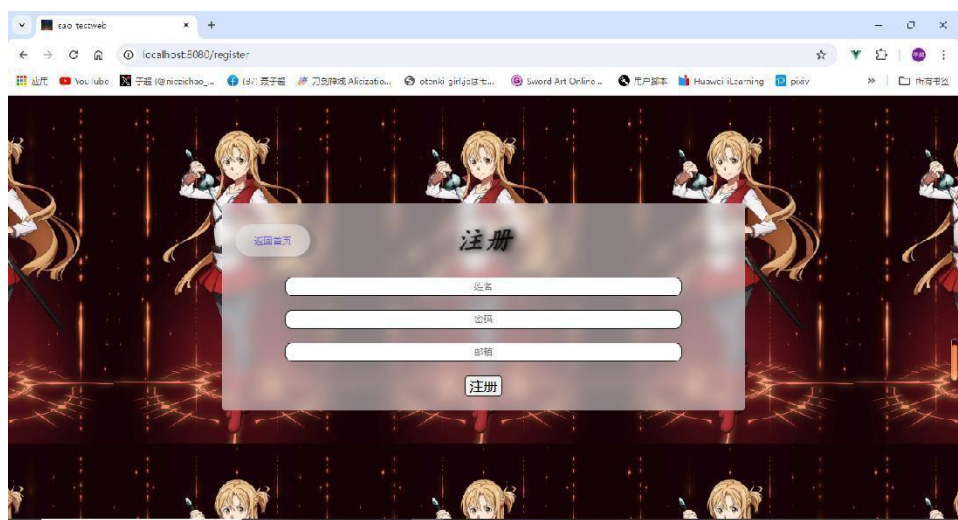
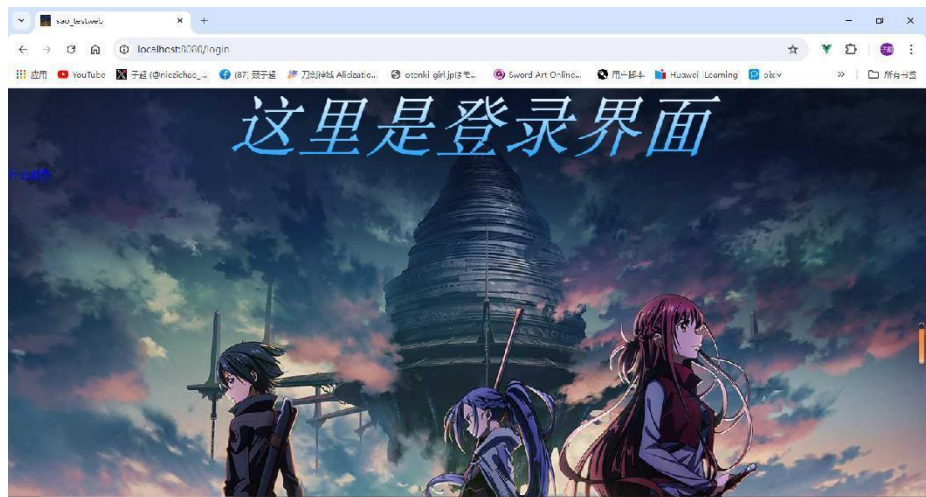


当前 vue(截止目前，目前尝试新增注册登录跳转)

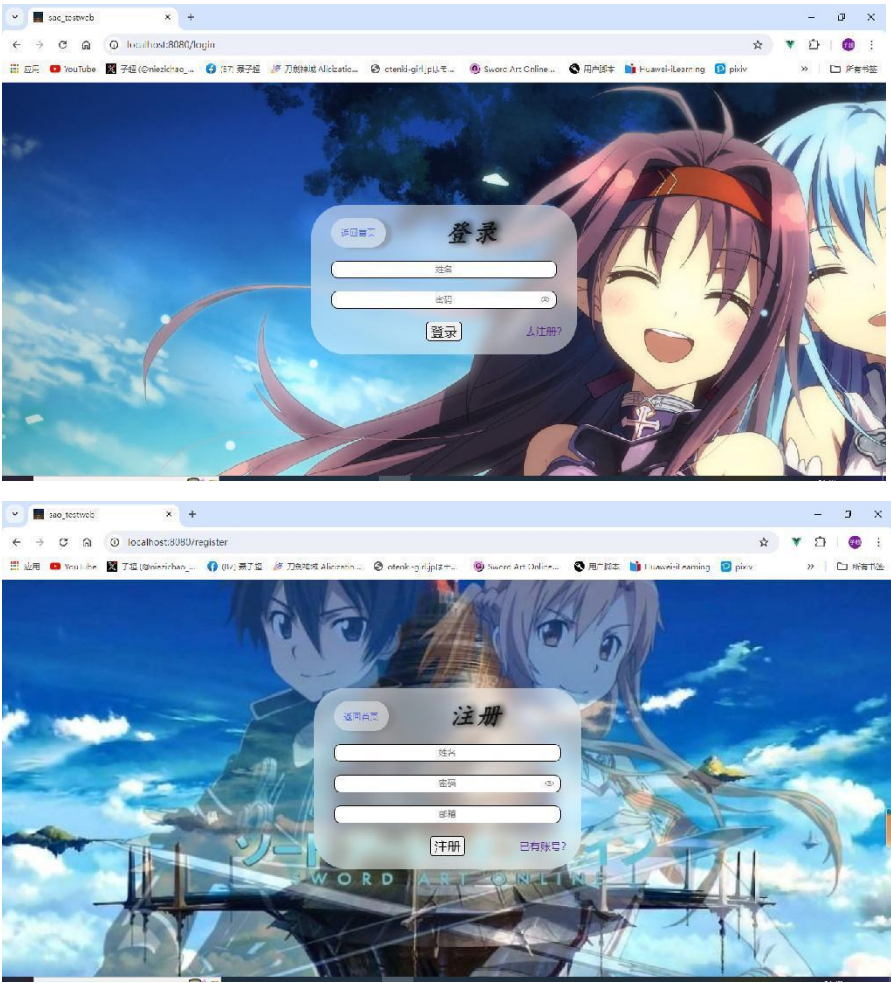




登录注册页面(暂时)

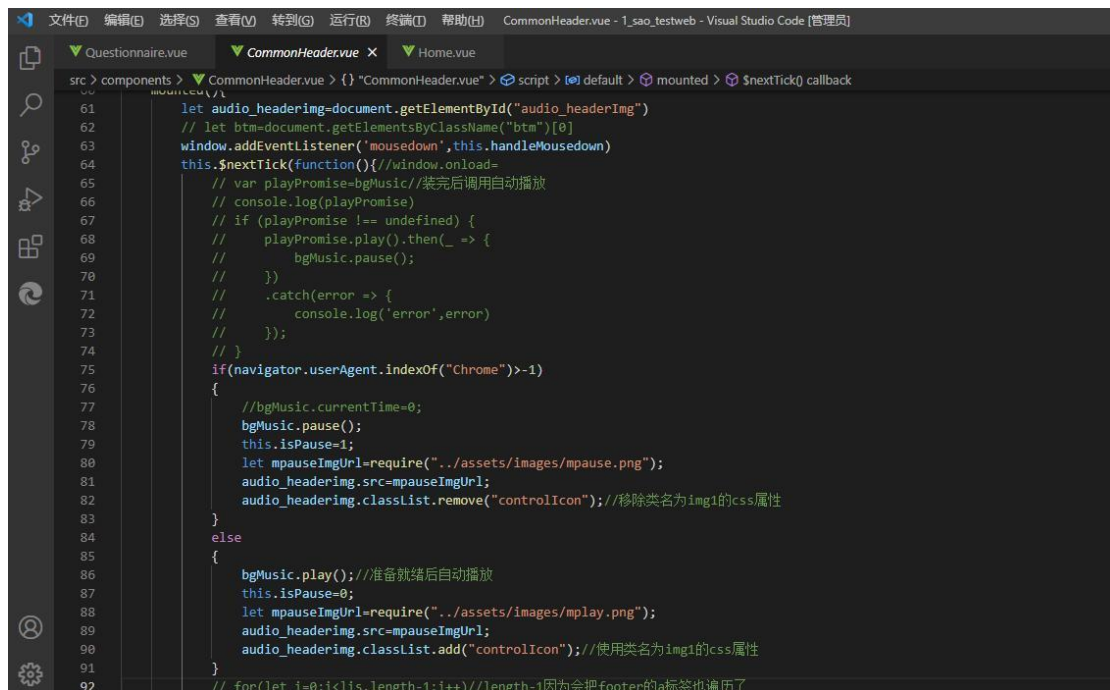


登录注册页面(目前)



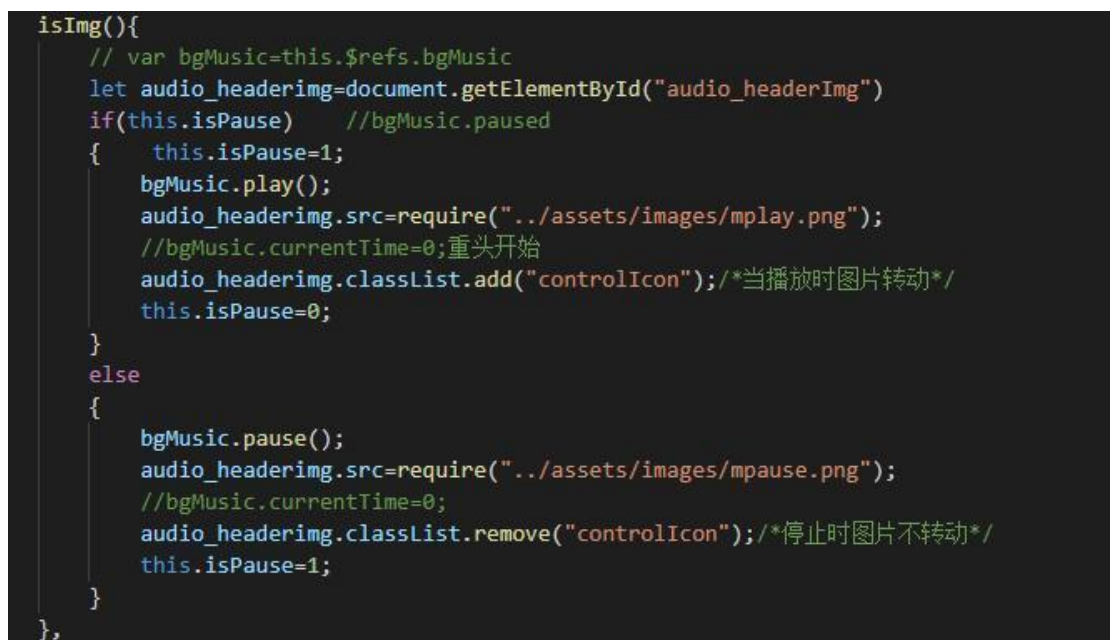
原 html 中部分功能代码（详细请查看 web 期末作业项目
readme）：

判断浏览器内核类型实现背景音乐是否自动播放



```
src > components > CommonHeader.vue > {} "CommonHeader.vue" > script > default > mounted > $nextTick() callback
61 let audio_headerimg=document.getElementById("audio_headerImg")
62 // let btm=document.getElementsByClassName("btm")[0]
63 window.addEventListener('mousedown',this.handleMousedown)
64 this.$nextTick(function(){//window.onload=
65 // var playPromise=bgMusic//装完后调用自动播放
66 // console.log(playPromise)
67 // if (playPromise !== undefined) {
68 //   playPromise.play().then(_ => {
69 //     bgMusic.pause();
70 //   })
71 //   .catch(error => {
72 //     console.log("error",error)
73 //   });
74 // }
75 if(navigator.userAgent.indexOf("Chrome")>-1)
76 {
77   //bgMusic.currentTime=0;
78   bgMusic.pause();
79   this.isPause=1;
80   let mpauseImgUrl=require("../assets/images/mpause.png");
81   audio_headerimg.src=mpauseImgUrl;
82   audio_headerimg.classList.remove("controlIcon");//移除类名为img1的css属性
83 }
84 else
85 {
86   bgMusic.play();//准备就绪后自动播放
87   this.isPause=0;
88   let mplayImgUrl=require("../assets/images/mplay.png");
89   audio_headerimg.src=mplayImgUrl;
90   audio_headerimg.classList.add("controlIcon");//使用类名为img1的css属性
91 }
92 // for(let i=0;i<lis.length-1;i++)//length-1因为会把footer的a标签也遍历了
```

点击图标时停止旋转图标同时暂停背景音乐，重新点击后再次旋转并重新播放音乐：



```
isImg(){
  // var bgMusic=this.$refs.bgMusic
  let audio_headerimg=document.getElementById("audio_headerImg")
  if(this.isPause) //bgMusic.paused
  {
    this.isPause=1;
    bgMusic.play();
    audio_headerimg.src=require("../assets/images/mplay.png");
    //bgMusic.currentTime=0;重头开始
    audio_headerimg.classList.add("controlIcon");//当播放时图片转动*/
    this.isPause=0;
  }
  else
  {
    bgMusic.pause();
    audio_headerimg.src=require("../assets/images/mpause.png");
    //bgMusic.currentTime=0;
    audio_headerimg.classList.remove("controlIcon");//停止时图片不转动*/
    this.isPause=1;
  }
},
```

通过学习 vue 等前端所需技术知识后，将本科时期的 web 期末作品中的部分 html 代码使用 vue 重新构建。以下为 web 期末作品的代码说明以及部分 html 修改为 vue 的详情：

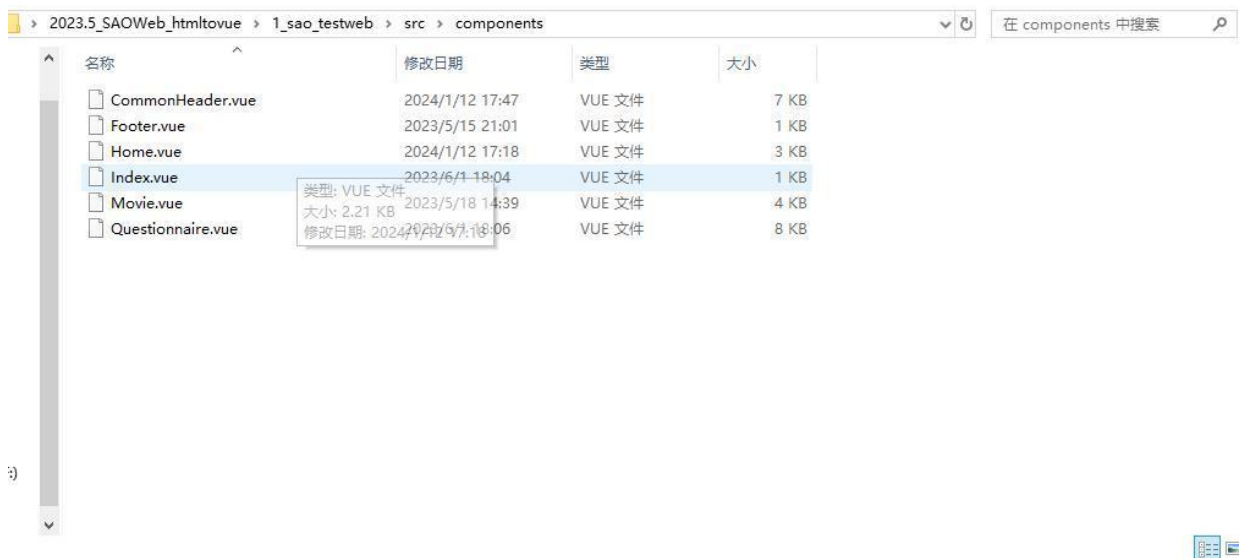
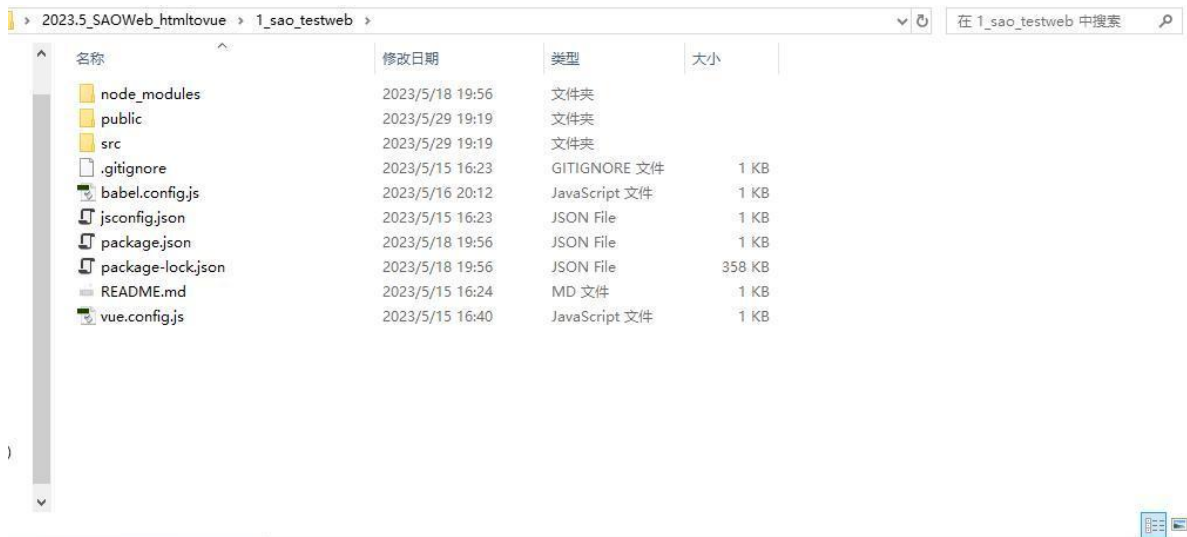
链接：<https://pan.baidu.com/s/18tfPaChGCe3OIUTKVVtI0w?pwd=l61z>

提取码：l61z (若图片不清楚可查看该链接)

部分 html 改为 vue:

由单个 html 文件改为多个组件组成





导航栏中的 a 标签改为 vue 中的 router-link 标签:



声明一个变量指向 data 中返回的 ruleForm

```
export default {
  name: 'Questionnaire',
  data() { // 使用组件时必须使用函数式
    return {
      // homeUrl: '/home',
      ruleForm: {
        isage: '',
        isemail: '',
        checkboxes: [],
        sex: '',
        interest: '',
        istextarea: ''
      }
    }
  },
}
```

```
let ruleForm=this.ruleForm;
```

判断问卷内容必填项是否填完

```
//验证判断
if( istrue && phone.value.length >=8
  && phone.value.length <=11 && ruleForm.istextarea !== '' )
{
  alert('感谢您的反馈! 正在返回首页! ')
  // document.getElementById ('sub').click()
  this.$router.push({name: 'home', params: ruleForm});
}
else{
  alert("请检查输入的信息!"); return
}
```

跳转并把数据传到首页

```
data(){
  return{
    isShow: false,
    mess: this.$route.params
  }
},
```

嵌套，动态渲染

```
<template>
  <div>
    <CommonHeader></CommonHeader>
    <!--section-->
    <router-view></router-view>
    <!--//section-->
    <!--footer-->
    <Footer></Footer>
    <!--//footer-->
  </div>
</template>
```

2024.5.24 新增:

勾选单选框并获取绑定对应标签的文本内容:

```
let sexchoose = document.getElementsByName('sex');

sexchoose.forEach(item=>{
  if(item.checked){
    //判断是否被选中后获取绑定的label
    var sexlabel=document.querySelector(
      'label[for="'+ item.id + '"]');
    ruleForm.sex=sexlabel.innerText;
  }
});
```

勾选复选框并获取绑定对应标签的文本内容:

```
let checkboxes = document.getElementsByName('interest');
let istrue = false; //复选框默认都没勾选
```

```
let temp=[];
```

```
//遍历选中复选框
for(let i = 0; i < checkboxes.length; i++){
  if(checkboxes[i].checked){
    //任意一个复选框被勾选
    istrue = true;
    //判断是否被选中后获取绑定的label
    var interest = document.querySelector(
      'label[for="'+ checkboxes[i].id + '"]').innerText;
    temp.push(interest);
  }
}
ruleForm.interest=(temp.toString().split(','));
```


Textarea 内容输出到首页显示信息窗口时限制每行字符数:

```
//textarea每行限制输出显示长度
let txttemp=[];
// 转变为数组形式
let textArr = ruleForm.istextarea.split('');
for(let i = 0;i<textArr.length;i++){
    if(i%50==49){//每50个字符换行
        txttemp.push(textArr[i] += '\n');
    }else{//50以内直接插入数组
        txttemp.push(textArr[i]);
    }
}
// 数组转变为字符串形式
ruleForm.istextarea = txttemp.join('');
```

2024.5.29 修改:

将联系号码的 input 标签中的最小长度限制删除, 并在验证判断中添加手机号码格式验证:

```
// 判断输入的手机格式是否正确
let telstyle = /^1[3-9][0-9]{9}$/;
if(telstyle.test(ruleForm.istel)){
    // alert(ruleForm.istel);
    alert('感谢您的反馈! 正在返回首页! ')
    // document.getElementById ('sub').click()
    this.$router.push({name:'home',params:ruleForm});
}else{
    alert('手机格式错误! ');
    ruleForm.istel='';//清除错误内容
    phone.focus();
}
```

2024.6.20 补全:

Textarea 内容输出到首页显示信息窗口时限制每行字符数（由直接判断修改成按类型判断）:

```
162 //textarea每行限制输出显示长度
163 let txttemp=[];
164 // 转变为数组形式
165 let textArr = ruleForm.istextarea.split(''); //推荐let temptArr=ruleForm.istextarea; let textArr=temptArr.split('');
166 // console.log('旧的: ',textArr)
167 for(let i = 0;i<textArr.length;i++){
168     let reg = /[u4e00-u9fa5]/; //判断是否有汉字
169     if(textArr[i-1]!='\n'){ //textarea中原先已有'\n'
170         txttemp.push('\xa0\xa0\xa0\xa0\xa0\xa0\xa0\xa0');
171         // txttemp.push(textArr[i]);
172     }
173     let temp = i;
174     for(let j = 0,k = 0,l=0;j<(textArr.length-i);j++,temp++){ //原有\n即开始第二段 此处遍历外部for循环中的i值固定
175         // arr[j]=textArr[temp]; //arr=[]新段落的索引从0开始排序
176         if(reg.test(textArr[temp])){//是汉字(2个字符)
177             // console.log('arr[${j}]:',reg.test(arr[j]),arr[j],'arr[${j-1}]:',/[A-z0-9]/.test(arr[j-1]),arr[j-1],arr[j-1])
178             //判断当前汉字的相邻是否为汉字,如果不是则换行!reg.test(textArr[i-1]) &&
179             if(/[A-z0-9]/.test(textArr[temp-1]) && textArr[temp-1] !== undefined){ //避免arr[-1]=undefined的情况(j-1>=0)
180                 //\w:[A-z0-9]任意数字、字母 xa0英文空格
181                 txttemp.push('\n\xa0\xa0\xa0\xa0\xa0\xa0\xa0\xa0\xa0\xa0');
182                 txttemp.push(textArr[temp]); //换行后插入第一个汉字
183             }else{
184                 if(i%12==9){ //每12个汉字(22个字符)换行
185                     txttemp.push(textArr[temp]);
186                     txttemp.push('\n');
187                 }else{//12以内直接插入数组
188                     txttemp.push(textArr[temp]);
189                 }
190                 k++;l=0;
191             }else{//不是汉字
192                 //k为非汉字时的索引,从0开始计算
193                 if(!(/[A-z]/.test(textArr[temp])&&/[A-z]/.test(textArr[temp-1])) ||
194                     (!/[0-9]/.test(textArr[temp])&&/[0-9]/.test(textArr[temp-1])) || j==0){
195                     //都是同类型时
196                     if(i%20==15){ //每20个字符换行
197                         txttemp.push(textArr[temp]);
198                         txttemp.push('\n');
199                     }else{//20以内直接插入数组
200                         txttemp.push(textArr[temp]);
201                     }
202                 }else if(textArr[temp]=='\n'){
203                     txttemp.push(textArr[temp]);
204                     txttemp.push('\xa0\xa0\xa0\xa0\xa0\xa0\xa0');
205                 }else if(txttemp[txttemp.length-1]=='\n'){
206                     //新数组最后一个为\n或者遇到换行
207                     txttemp.push('\xa0\xa0\xa0\xa0\xa0\xa0\xa0\xa0');
208                     txttemp.push(textArr[temp]);
209                 }else{//前后不是同类型且前面非'\n'时
210                     txttemp.push('\n\xa0\xa0\xa0\xa0\xa0\xa0\xa0\xa0\xa0');
211                     txttemp.push(textArr[temp]);
212                 }
213                 l++;k=0;
214             }
215         }
216     }
217     break; //遍历完整段后跳出循环
218 }else{//textarea中原先没有'\n'
219     if(reg.test(textArr[i])){//是汉字(2个字符)
220         //判断当前汉字的前面是否为汉字,如果不是则换行!reg.test(textArr[i-1]) &&
221         if(/[A-z0-9]/.test(textArr[i-1])){
222             //\w:[A-z0-9]任意数字、字母 xa0英文空格
223             txttemp.push('\n\xa0\xa0\xa0\xa0\xa0\xa0\xa0\xa0\xa0\xa0');
224             txttemp.push(textArr[i]); //换行后插入第一个汉字
225         }else{
226             if(i%12==9){ //每12个汉字(22个字符)换行
227                 txttemp.push(textArr[i]);
228                 txttemp.push('\n');
229             }else{//12以内直接插入数组
230                 txttemp.push(textArr[i]);
231             }
232         }
233     }else{//不是汉字
234         if(!(/[A-z]/.test(textArr[i])&&/[A-z]/.test(textArr[i-1])) ||
235             (!/[0-9]/.test(textArr[i])&&/[0-9]/.test(textArr[i-1])) || i==0){
236             //都是同类型时
237             if(i%20==15){ //每20个字符换行
238                 txttemp.push(textArr[i]);
239                 txttemp.push('\n');
240             }else{//20以内直接插入数组
241                 txttemp.push(textArr[i]);
242             }
243         }else if(textArr[i]=='\n'){
```

```

232     }else{//不是汉字
233         if((/[A-z]/.test(textArr[i])&&/[A-z]/.test(textArr[i-1])) ||
234             ([0-9]/.test(textArr[i])&&/[0-9]/.test(textArr[i-1])) || i==0){
235             //都是同类型时
236             if(i%20==15){//每20个字符换行
237                 txttemp.push(textArr[i]);
238                 txttemp.push('\n');
239             }else{//20以内直接插入数组
240                 txttemp.push(textArr[i]);
241             }
242         }else if(textArr[i]==='\n'){
243             txttemp.push(textArr[i]);
244         }else if(txttemp[txttemp.length-1]==='\n'){
245             txttemp.push('\xa0\xa0\xa0\xa0\xa0\xa0');
246             txttemp.push(textArr[i]);
247         }else{//前后不是同类型且前面非'\n'时
248             txttemp.push('\n\xa0\xa0\xa0\xa0\xa0\xa0');
249             txttemp.push(textArr[i]);
250         }
251     }
252 }
253 // console.log("新的: ",txttemp);
254 // 数组转变为字符串形式
255 ruleForm.istextarea = txttemp.join('');
256
257

```

因循环内相对判断较多且有重复语句，后续会将可拿出来的代码重复部分封装成函数再调用。

新增连接数据库(node.js 和 mongoose):

尝试将问卷中输入的内容，在按下提交按钮后将数据以 JSON 对象形式发送请求到 node.js，并实现跨域

```

//验证判断
if( ruleForm.isage<=150 && istrue && phone.value.length >=7
    && phone.value.length <=11 && ruleForm.istextarea !== '' )
{
    // 判断输入的手机格式是否正确
    let telstyle = /^1[3-9][0-9]{9}$/;
    if(telstyle.test(ruleForm.istel)){
        //1.发送axios请求
        axios.post('http://localhost:8080/home',ruleForm,//JSON.stringify(ruleForm),将JSON对象变成字符串
            //其他配置
            {
                headers:{
                    // //application/x-www-form-urlencoded:最常见的 POST 提交数据的方式了。浏览器的原生表单
                    'Content-Type': 'application/json',
                }
            }
        ).then(res=>{
            alert('感谢您的反馈! 正在返回首页! ');
            this.$router.push({name:'home',params:ruleForm});
        }).catch(err=>{
            console.error(err);
        });
        // document.getElementById('sub').click()
        // this.$router.push({path:'/home',query:{key:'传达到了'}});

        //2.调用ajax请求
        // this.sendxhr(this,ruleForm);
    }
}

```



```

1 //导入express
2 const express = require('express');
3 //解决跨域, 导入中间件cors
4 const cors = require('cors');
5 // 处理类型是 application/x-www-form-urlencoded 和 application/json 格式的表单
6 // const bodyParser = require('body-parser');//express4.16+已弃用body-parser
7 //创建应用对象
8 const app = express();
9 //注册跨域中间件
10 app.use(cors());
11 //解析querystring格式请求体的中间件
12 // const urlencodedParser = bodyParser.urlencoded({
13 //   extended:false//false 表示使用系统模块querystring来处理
14 // });
15 // 解析JSON格式的请求体
16 app.use(express.json());
17

```

```

app.post('/home', (req,res)=>{
  //设置响应头
  res.setHeader("Access-Control-Allow-Origin","*");
  res.setHeader("Access-Control-Allow-Headers","*");
  res.setHeader("Access-Control-Allow-Method","*");
  res.send('触发了这个post方法');
  console.log(req.body);
});

```

尝试将问卷中输入的内容,在按下提交按钮后将数据以 JSON 对象形式发送请求到 mongoose 数据库中

Node.js 文件中添加关于 mongoose 的 post 请求

```

//使用mongoose模块化]
const questionnaireModel = require('./model/questionnaireModel');
const mongodb = require('./db/mongodb');
// const questionnaireModel = require('./db/mgdb');
app.post('/home', (req,res,next)=>{
  //设置响应头
  res.setHeader("Access-Control-Allow-Origin","*");
  res.setHeader("Access-Control-Allow-Headers","*");
  res.setHeader("Access-Control-Allow-Method","*");
  mongodb(()=>{
    questionnaireModel.create({
      ...req.body
    }).then(data=>{
      res.json({
        code:'0000',
        msg:'创建成功',
        data:data
      });
    }).catch(err=>{
      res.json({
        code:'0001',
        msg:'创建失败',
        data:null
      });
    });
    return;
  });
});
});

```

新建数据库连接文件 mongodb.js

```
4  * @param {*} error 数据库连接失败的回调
5  */
6  module.exports=function(success,error){
7    //判断error,为其设置默认值
8    if(typeof error !== 'function'){
9      error = ()=>{};
10     console.log('连接失败!');
11   };
12 }
13 const express = require('express');
14 //解决跨域,导入中间件cors
15 const cors = require('cors');
16 //1.安装mongoose
17 //2.导入mongoose
18 const mongoose = require('mongoose');
19 const app = express();
20 //注册跨域中间件
21 app.use(cors());
22 // 解析JSON格式的请求体
23 app.use(express.json());
24 //设置strictQuery(默认严格查询模式)为true
25 mongoose.set('strictQuery',true);
26 //3.连接mongodb服务 端口后是数据库名称(数据库无则自动创建)
27 mongoose.connect('mongodb://127.0.0.1:27017/saotestweb');
28 //4.设置回调
29 //连接成功的回调
30 mongoose.connection.once('open',()=>{
31   //当mongodb服务断线时,8080端口依然占用,若使用on,则会在服务重新上线时重连,但无法监听8080端口
32   console.log('连接成功!');
33   success();
34 });
35 //连接失败的回调
```

新建 Mongoose 模块文件 questionnaireModel.js

```
const mongoose = require('mongoose');
//1.创建文档的结构对象
let questionnaireSchema = new mongoose.Schema({
  sex:String,
  isage:Number,
  isemail:String,
  istel:Number,
  acg:String,
  interest:[String],
  istextarea:String
});
//2.创建模型对象 (集合名称, 结构对象)
let questionnaireModel = mongoose.model('ruleForm',questionnaireSchema);

module.exports = questionnaireModel;
```

将问卷内容填写后通过 axios 请求将数据发送到数据库并添加至数据库中



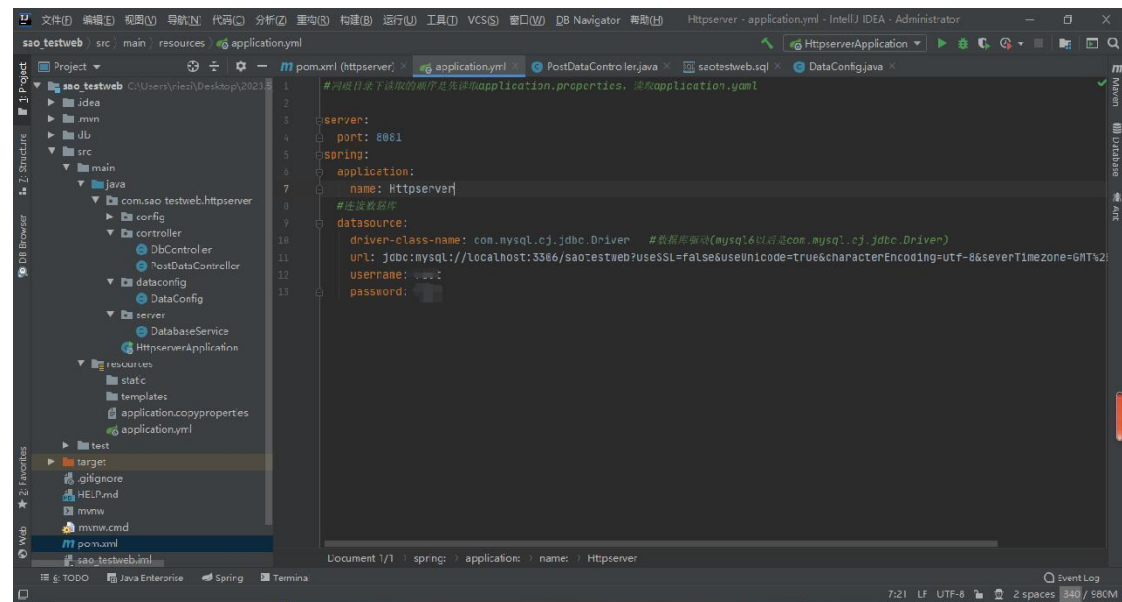
集合	视图	函数	索引	MapReduce	GridFS	用户	查询	自动运行	模型
对象	ruleforms	@saotestweb (local)	集合						
开始事务	文本	筛选	排序	全部折叠	类型颜色	导入	导出	分析	
_id	sex	isage	isemail	istel	acg	interest			
66868ca6bed34a212ec035f4	男	1	12@xx.com		接触过	(Array) 1			
668bd2f8ef958a4333703922	男	1	r		接触过	(Array) 1			
668bd32def958a4333703924	男	2	n		接触过	(Array) 1			
668c01a1e05807bee4b4a749	男	132	12@xx.com		接触过	(Array) 1			
668c025cde3f6fb52c451833	男	132			接触过	(Array) 1			
668c0a61b50a925fff738ffe	男	412	12@xx.com		接触过	(Array) 1			
668c0b63ba77cf02f3557e79	男	412	12@xx.com		接触过	(Array) 1			
668c0beb85a758cdc4da8fbc	男	555	12@xx.com		接触过	(Array) 1			

2024.8.30 补全:

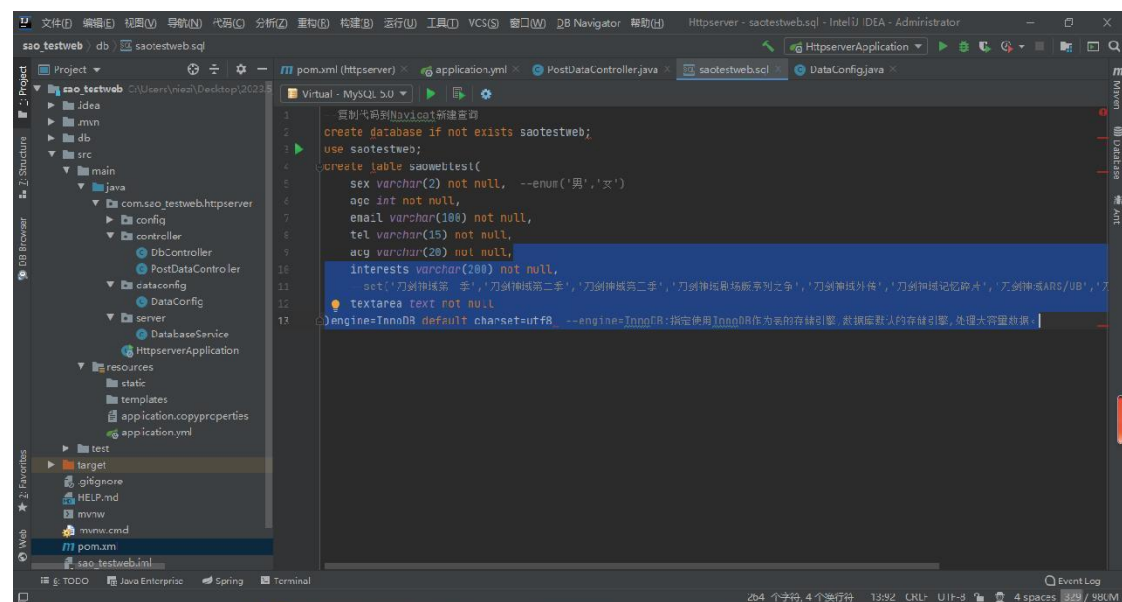
2024.7.26 连接 mysql 数据库并写入相关数据(从问卷中提交的数据)

Java 部分:

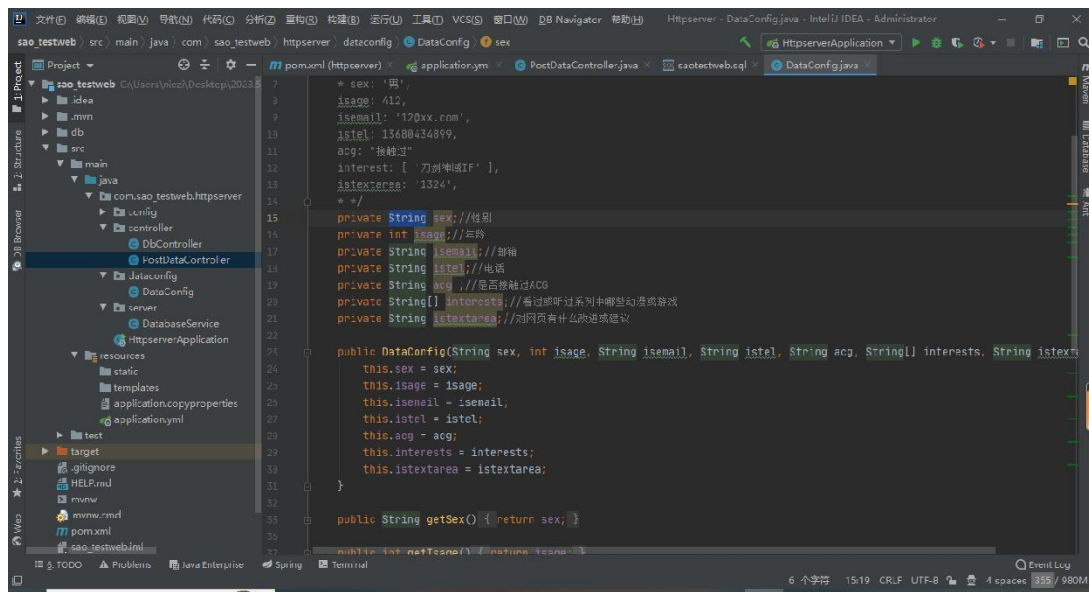
application.yml 文件:



saotestweb.sql 文件:

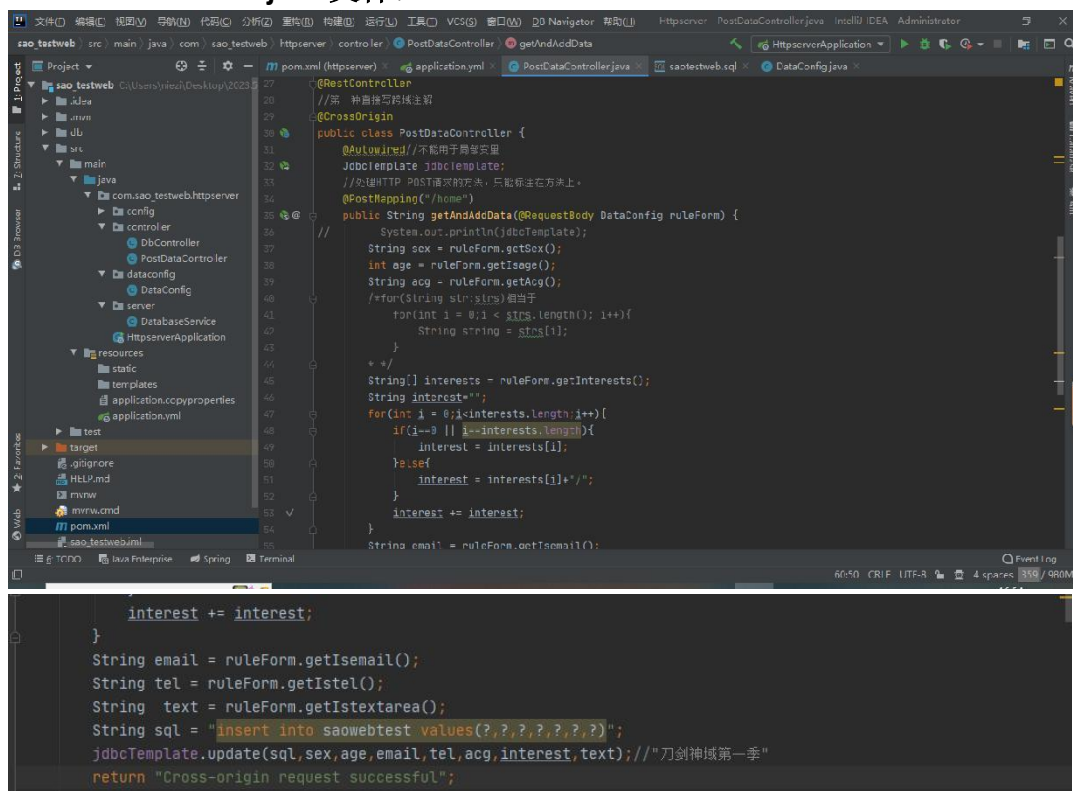


DataConfig.java 文件:



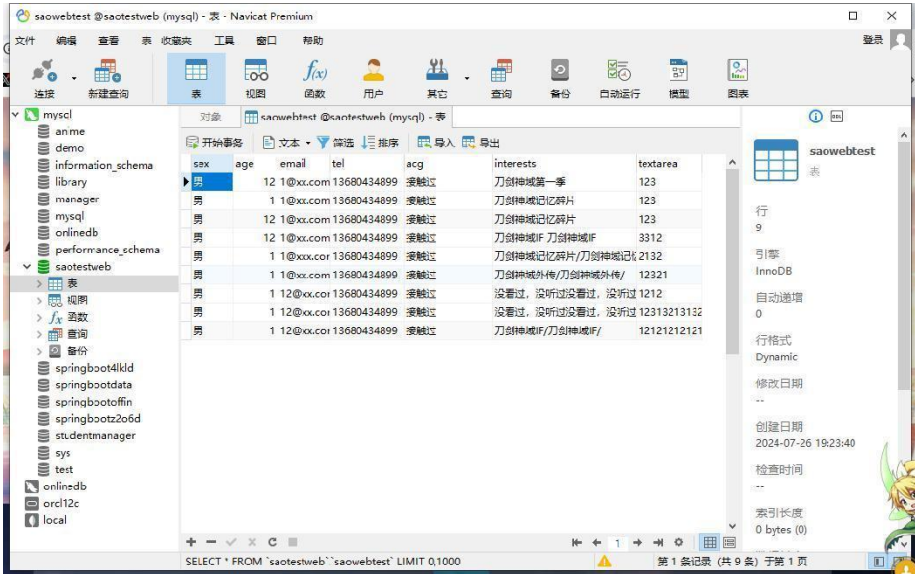
```
7  * sex: '男',
8  isage: 412,
9  isemail: '120xx.com',
10 istel: 13688434899,
11 acg: "张赫迪"
12 interest: [ '刀剑神域IF' ],
13 istextarea: '1324',
14 * */
15 private String sex; //性别
16 private int isage; //年龄
17 private String isemail; //邮箱
18 private String istel; //电话
19 private String acg; //是否接触过ACG
20 private String[] interests; //看过听过系列中哪些动漫或游戏
21 private String istextarea; //对网页有什么的建设或建议
22
23 public DataConfig(String sex, int isage, String isemail, String istel, String acg, String[] interests, String istextarea) {
24     this.sex = sex;
25     this.isage = isage;
26     this.isemail = isemail;
27     this.istel = istel;
28     this.acg = acg;
29     this.interests = interests;
30     this.istextarea = istextarea;
31 }
32
33 public String getSex() { return sex; }
```

PostDataController.java 文件:



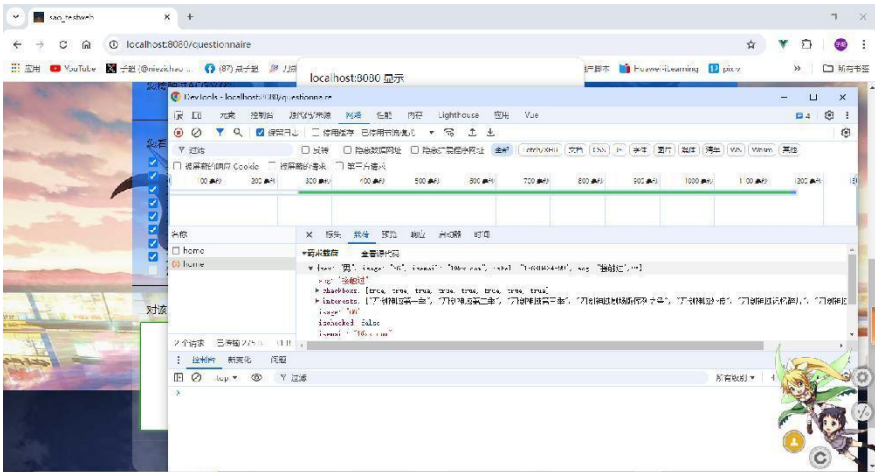
```
27 @RestController
28 //第 一种直接写跨域注解
29 @CrossOrigin
30 public class PostDataController {
31     @Autowired //不能用于局部变量
32     JdbcTemplate jdbcTemplate;
33     //处理HTTP POST请求的类，只能标注在类上。
34     @PostMapping("/home")
35     public String getAndAddData(@RequestBody DataConfig ruleForm) {
36         System.out.println(jdbcTemplate);
37         String sex = ruleForm.getSex();
38         int age = ruleForm.getIsage();
39         String acg = ruleForm.getAcg();
40         //for(String str:interests)相当于
41         for(int i = 0; i < interests.length(); i++){
42             String string = interests[i];
43         }
44         String[] interests = ruleForm.getInterests();
45         String interest="";
46         for(int i = 0; i < interests.length; i++){
47             if(i==0 || i==interests.length-1){
48                 interest = interests[i];
49             }else{
50                 interest = interests[i]+" ";
51             }
52             interest += interests[i];
53         }
54         String email = ruleForm.getIsemail();
55
56         String sql = "insert into saowebtest values(?,?,?, ?, ?, ?, ?)";
57         jdbcTemplate.update(sql, sex, age, email, tel, acg, interest, text); // "刀剑神域第一季"
58         return "Cross-origin request successful";
59     }
60 }
```


添加新数据前 Navicat 可视化表中数据为:

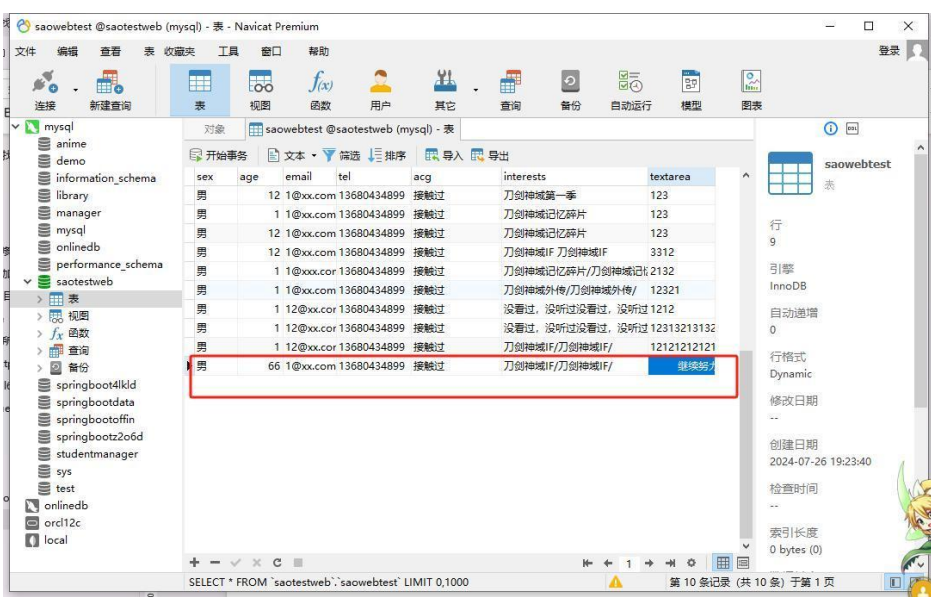


sex	age	email	tel	acq	interests	textarea
男	12	1@xx.com	13680434899	接触过	刀剑神域第一季	123
男	1	1@xx.com	13680434899	接触过	刀剑神域记忆碎片	123
男	12	1@xx.com	13680434899	接触过	刀剑神域记忆碎片	123
男	12	1@xx.com	13680434899	接触过	刀剑神域IF 刀剑神域IF	3312
男	1	1@xx.com	13680434899	接触过	刀剑神域记忆碎片/刀剑神域记忆	2132
男	1	1@xx.com	13680434899	接触过	刀剑神域外传/刀剑神域外传/	12321
男	1	12@xx.com	13680434899	接触过	没看过, 没听过没看过, 没听过	1212
男	1	12@xx.com	13680434899	接触过	没看过, 没听过没看过, 没听过	1231321312
男	1	12@xx.com	13680434899	接触过	刀剑神域IF/刀剑神域IF/	12121212121

添加新数据后
F12 控制台工具:



Navicat 可视化表中数据为:



sex	age	email	tel	acq	interests	textarea
男	12	1@xx.com	13680434899	接触过	刀剑神域第一季	123
男	1	1@xx.com	13680434899	接触过	刀剑神域记忆碎片	123
男	12	1@xx.com	13680434899	接触过	刀剑神域记忆碎片	123
男	12	1@xx.com	13680434899	接触过	刀剑神域IF 刀剑神域IF	3312
男	1	1@xx.com	13680434899	接触过	刀剑神域记忆碎片/刀剑神域记忆	2132
男	1	1@xx.com	13680434899	接触过	刀剑神域外传/刀剑神域外传/	12321
男	1	12@xx.com	13680434899	接触过	没看过, 没听过没看过, 没听过	1212
男	1	12@xx.com	13680434899	接触过	没看过, 没听过没看过, 没听过	1231321312
男	1	12@xx.com	13680434899	接触过	刀剑神域IF/刀剑神域IF/	12121212121
男	66	1@xx.com	13680434899	接触过	刀剑神域IF/刀剑神域IF/	12121212121